Git & GitHub: The Essentials

Git is a free and open-source distributed version control system (VCS) designed to handle everything from small to very large projects with speed and efficiency.

It allows you to track changes to your files, revert to previous states, and collaborate with others without overwriting each other's work.

1. The Three States (The Git Workflow)

Git manages files in three main areas. Understanding this workflow is the most critical step to mastering Git.

State	Definition	How to Interact
1. Working Directory	The files currently on your computer's filesystem. Where you make and edit files.	Edit code, save files.
2. Staging Area	A snapshot of the changes you want to include in your next commit. This is the **intermediate** area.	`git add <file>`
3. Git Directory (Repository)	Where Git stores the history of your project (the database). The permanent, saved version of your code.	`git commit`

The Basic Workflow Cycle

1. **Modify** files in the **Working Directory**.
2. **Stage** the files (`git add`) to the **Staging Area**.
3. **Commit** the files (`git commit`) from the Staging Area to the **Git Directory**.

2. Initial Setup & Configuration

Before you start, tell Git who you are. This information is attached to every commit you make.

Command	Purpose	Example
`git config --global user.name`	Sets your name for all future commits.	`git config --global user.name "Seif Ahmed"`
`git config --global user.email`	Sets your email for all future commits.	`git config --global user.email "seif@example.com"`
`git config --list`	Displays all configuration settings.	

3. Local Repository Commands

These commands are used to manage the history and state of your project on your local machine.

A. Initialization

Command	Purpose
`git init`	Initializes a new local Git repository in the current directory. This creates the hidden `.git` folder.

B. Status & Tracking

Command	Purpose
`git status`	Shows the state of the working directory and the staging area (which files are modified, staged, or untracked).
`git add <file_name>`	Puts a file into the Staging Area, preparing it for a commit.
`git add .`	Stages **all** modified and new files in the current directory.
`git rm --cached <file_name>`	Unstages a file without deleting it from the working directory.

C. Committing

Command	Purpose
`git commit -m "Commit Message"`	Records a snapshot of the Staging Area permanently into the Git repository history. The message (`-m`) is mandatory.

D. History

Command	Purpose
`git log`	Shows the commit history (commit ID, author, date, and message).
`git log --oneline`	Shows a condensed, single-line summary of the history.

4. Remote Repository Commands

These commands are used to interact with a remote repository, typically hosted on platforms like GitHub, GitLab, or Bitbucket.

A. Getting Started (First Time)

Command	Purpose
`git clone <URL>`	Downloads an existing remote repository to your local machine.
`git remote add origin <URL>`	Links your local repository to a remote repository URL. `origin` is the default name for the remote.

B. Synchronization

Command	Purpose
`git pull origin <branch_name>`	**Fetch** changes from the remote repository and **Merge** them into your current branch.
`git push origin <branch_name>`	Sends your local committed changes to the remote repository.

C. Branching (For Collaboration)

Branching allows developers to work on features in isolation from the main codebase.

Command	Purpose
`git branch`	Lists all local branches.
`git branch <new_branch>`	Creates a new branch.
`git checkout <branch_name>`	Switches to the specified branch (pre-2.23 version).
`git switch <branch_name>`	Switches to the specified branch (preferred modern command).
`git merge <branch_name>`	Integrates changes from the specified branch into the current branch.

Quick Start Guide

1. **Initialize:** `git init`
2. **Create/Edit:** Add your files (`index.html` , `style.css` , etc.)
3. **Check Status:** `git status`
4. **Stage Files:** `git add .`
5. **Commit Changes:** `git commit -m "Initial commit of project structure"`
6. **Push to Remote:** (First time setup needed) `git push origin main`